Assignment-more-epochs

November 21, 2025

[35]:
```
%reload_ext autoreload
%autoreload 2
```

[36]:
```
import torch
import torch.nn as nn
from torch.nn import functional as F
import math
from torch.nn import LayerNorm
```

[37]:
```
from kret_studies import *
from kret_studies.notebook import *
from kret_studies.complex import *
```

[38]:
```
from kret_studies.kret_torch.mixin.constants import DEVICE_TORCH_STR,␣
↪DEVICE_LITERAL

device: DEVICE_LITERAL = DEVICE_TORCH_STR
NUM_EPOCHS = 1
```

[39]:
```
from trainer import save_model_auto, load_model_auto, model_filename
```

[40]:
```
# !pip install thop
# !pip install tdqm
```

1 Part 1: Attention and Encoder-Decoder Models

In this section, you'll implement two forms of attention and use them in an encoder-decoder style transformer. You'll then compare performance based on changing dataset size and hidden dimension.

1.1 Step 1: Scaled Dot Product Attention

Implement both the base `scaled_dot_attention` method as well as the `forward` method for the `Attention` module.

[41]:
```
def scaled_dot_attention(
    q: torch.Tensor, k: torch.Tensor, v: torch.Tensor, mask: torch.Tensor |␣
↪None = None
```

```
):
    """Computes scaled dot product attention with an optional mask.
      q : shape ( batch, k, hidden_size)
      k : shape ( batch, seq_len, hidden_size)
      v : shape ( batch, seq_len, hidden_size)
      mask: optional. shape (k, seq_len)

    returns:
        context   : shape (batch, k, hidden_size)
        attention : shape (batch, k, seq_len)
    """

    # ------------
    # FILL THIS IN
    # ------------
    unnorm_attn = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(q.size(-1))
    if mask is not None:
        unnorm_attn = unnorm_attn.masked_fill(mask == 0, float("-inf"))
    attention = F.softmax(unnorm_attn, dim=-1)
    context = torch.matmul(attention, v)
    return context, attention
```

[42]:
```
class Attention(nn.Module):
    def __init__(self, hidden_size: int):
        super().__init__()
        self.Q = nn.Linear(hidden_size, hidden_size, bias=False)
        self.K = nn.Linear(hidden_size, hidden_size, bias=False)
        self.V = nn.Linear(hidden_size, hidden_size, bias=False)
        self.hidden_size = hidden_size
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor, annots: torch.Tensor):
        #   x         : shape ( batch, k, hidden_size)
        #   annots    : shape ( batch, seq_len, hidden_size)

        # ------------
        # FILL THIS IN
        # Pay attention to which inputs you use for the queries, keys, and␣
 ↪values.
        # ------------
        # q =
        # k =
        # v =
        q = self.Q(x)
        k = self.K(annots)
        v = self.V(annots)
```

```
        return scaled_dot_attention(q, k, v)
```

1.2 Step 2: Causal Scaled Dot Product Attention

Using your `scaled_dot_attention` method, implement the `forward` method for the `CausalAttention` module.

[43]:
```
class CausalAttention(Attention):
    def __init__(self, hidden_size: int):
        super().__init__(hidden_size)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor):
        #   x : shape ( batch, seq_len, hidden_size)

        # ------------
        # FILL THIS IN
        # This should be very similar to your other attention implementation,
        # with the exception that you only have one set of outputs and you
        # must construct a causal attention mask.
        # ------------
        # q =
        # k =
        # v =
        # mask =
        seq_len = x.size(1)
        q = self.Q(x)
        k = self.K(x)
        v = self.V(x)
        mask = torch.tril(torch.ones((seq_len, seq_len), device=x.device)).
↪bool()
        return scaled_dot_attention(q, k, v, mask)
```

Simple test cases

Passing this DOES NOT mean your implementation is correct. But, if this fails, then you definitely did something wrong

[44]:
```
cros_attn = Attention(32)
caus_attn = CausalAttention(32)
caus_attn.Q = cros_attn.Q
caus_attn.K = cros_attn.K
caus_attn.V = cros_attn.V

inp = torch.randn(4, 8, 32).to(device)
inp2 = torch.randn(4, 16, 32).to(device)
```

```
print("Checking consistent shape")
assert cros_attn(inp, inp2)[0].shape == inp.shape
assert caus_attn(inp)[0].shape == inp.shape
assert cros_attn(inp, inp2)[1].shape == (4, 8, 16)
assert caus_attn(inp)[1].shape == (4, 8, 8)

print("Checking attention mask")
assert torch.all(caus_attn(inp)[0][:, -1] == cros_attn(inp, inp)[0][:, -1])

print("Passed")
```

```
Checking consistent shape
Checking attention mask
Passed
```

Read through the following code. Make sure you understand what is going on. This code sets up the encoder-decoder architecture. It requires no additional code.

[45]:
```
class PositionalEncoding(nn.Module):
    pe: torch.Tensor

    def __init__(self, hidden_size: int, max_len: int = 1000):
        super().__init__()

        pos = torch.arange(max_len).float().unsqueeze(1)
        dim = torch.arange(hidden_size // 2).float().unsqueeze(0)

        div_term = torch.exp(-math.log(10000.0) * (2 * dim) / hidden_size)
        angle = pos * div_term

        pe = torch.zeros(max_len, hidden_size)
        pe[:, 0::2] = torch.sin(angle)
        pe[:, 1::2] = torch.cos(angle)

        self.register_buffer("pe", pe)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size, max_len=max_len)

    def forward(self, idx: int):
        return self.pe[idx]
```

[46]:
```
class MLP(nn.Module):

    def __init__(self, hidden_size: int):
        super().__init__()
        self.layer1 = nn.Linear(hidden_size, hidden_size, bias=False)
        self.layer2 = nn.Linear(hidden_size, hidden_size, bias=False)
```

```
        self.relu = nn.ReLU()
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x


class EncoderBlock(nn.Module):
    def __init__(self, hidden_size: int):
        super().__init__()
        self.norm1 = LayerNorm(hidden_size)
        self.norm2 = LayerNorm(hidden_size)
        self.attn = Attention(hidden_size)
        self.mlp = MLP(hidden_size)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor):
        context, attention = self.attn(self.norm1(x), self.norm1(x))
        x = x + context
        x = x + self.mlp(self.norm2(x))
        return x, attention


class DecoderBlock(nn.Module):
    def __init__(self, hidden_size: int):
        super().__init__()
        self.norm1 = LayerNorm(hidden_size)
        self.norm2 = LayerNorm(hidden_size)
        self.cros_attn = Attention(hidden_size)
        self.self_attn = CausalAttention(hidden_size)
        self.mlp = MLP(hidden_size)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor, annotation: torch.Tensor):
        context, self_attn = self.self_attn(self.norm1(x))
        x = x + context
        context, cros_attn = self.cros_attn(self.norm2(x), annotation)
        x = x + context
        x = x + self.mlp(self.norm2(x))
        return x, self_attn, cros_attn
```

[47]:
```
class TransformerEncoderDecoder(nn.Module):
    def __init__(
        self, vocab_size: int, hidden_size: int, num_layers: int, dataset_name:␣
↪str
    ):
        super().__init__()

        self.tok_emb = nn.Embedding(vocab_size, hidden_size)
        self.pos_emb = PositionalEncoding(hidden_size)

        self.encoder_blocks = nn.ModuleList(
            [EncoderBlock(hidden_size) for i in range(num_layers)]
        )
        self.decoder_blocks = nn.ModuleList(
            [DecoderBlock(hidden_size) for i in range(num_layers)]
        )

        self.head = nn.Linear(hidden_size, vocab_size, bias=False)
        self.head.weight = self.tok_emb.weight
        # weight tie, the final layer reuses the weights of the embedding layers
        # this reduces the total number of parameters

        self.apply(self._init_weights)  # initialize the weights to a small␣
↪number
        self.to(device)
        self.filename = model_filename(
            hidden_size=hidden_size,
            num_layers=num_layers,
            vocab_size=vocab_size,
            dataset_name=dataset_name,
        )

    def forward(self, inputs: torch.Tensor, annots: torch.Tensor):

        pos_x = torch.arange(
            0, inputs.shape[-1], device=inputs.device
        ).long()  # creates tensor([0, 1, 2, 3, ...])
        pos_y = torch.arange(
            0, annots.shape[-1], device=inputs.device
        ).long()  # creates tensor([0, 1, 2, 3, ...])

        x = self.tok_emb(inputs) + self.pos_emb(pos_x)
        y = self.tok_emb(annots) + self.pos_emb(pos_y)

        self_attn = []
        cros_attn = []
```

```
        for encode in self.encoder_blocks:
            y, _ = encode(y)

        for decode in self.decoder_blocks:
            x, s, c = decode(x, y)
            cros_attn.append(c)
            self_attn.append(s)

        return self.head(x), {"self_attn": self_attn, "cros_attn": cros_attn}

    def _init_weights(self, module: torch.nn.Module):
        if isinstance(module, nn.Linear):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                torch.nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
```

1.3 Step 3: Training the model

You'll now train a set of example models. This section requires no additional code from you, with the exception of setting the datapath. However, **you are required to analyze and include some of its output**.

[48]:
```
from dataset import EncoderDecoderDataset, DecoderDataset
from utils import save_loss_comparison_by_dataset,␣
 ↪save_loss_comparison_by_hidden
from torch.utils.data import DataLoader
from trainer import train
import torch.optim as optim


data_path = str("./")  # Change data_path to where the data is stored

encdec_ds_small = EncoderDecoderDataset(data_path, "pig_latin_small")
encdec_ds_large = EncoderDecoderDataset(data_path, "pig_latin_large")
```

[49]:
```
print("Dataset sample:")
print("Input:", encdec_ds_small.to_token(encdec_ds_small[0][0].tolist()))
print("Annotation:", encdec_ds_small.to_token(encdec_ds_small[0][1].tolist()))
print("Target:", encdec_ds_small.to_token(encdec_ds_small[0][2].tolist()))
```

```
Dataset sample:
Input: <SOS>eeblefay
Annotation: feeble
Target: eeblefay<EOS>
```

[50]:
```
# model = load_model_auto(
#     TransformerEncoderDecoder,
#     vocab_size=len(encdec_ds_small.idx_to_token),
#     hidden_size=16,
#     num_layers=4,
#     dataset_name="small",
# )
model = TransformerEncoderDecoder(
    vocab_size=len(encdec_ds_small.idx_to_token),
    hidden_size=16,
    num_layers=4,
    dataset_name="small",
)
```

[51]:
```
optimizer = optim.AdamW(model.parameters(), lr=1e-2)
scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)

results = train(
    model=model,
    dataset=encdec_ds_small,
    num_epochs=NUM_EPOCHS,
    optimizer=optimizer,
    scheduler=scheduler,
    device=device,
    batch_size=32,
)
save_model_auto(model)
```

```
Epoch 1 | Train loss: 2.608072 | Val loss 2.266578
```

[51]:
```
PosixPath('/Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/TransformerEncoderDecoder/50_epochs/Model_ds_namesmall_vo
cab30_hid16_layers4.pt')
```

[18]:
```
def encdec_generate(
    input: str,
    model: TransformerEncoderDecoder,
    dataset: EncoderDecoderDataset,
    max_len: int = 20,
):
    gen_string = ""
    idx = torch.tensor([[dataset.start_idx]]).to(device)
    annots = torch.tensor(
        [dataset.to_idx(input) + [dataset.end_idx] * (max(0, 12 - len(input)))]
    ).to(device)
    # pad the input string with <EOS> since the collate_fn had to pad the␣
 ↪stirngs
```

```
    for i in range(max_len):
        logits = model(idx, annots)[0]

        token_idx: torch.Tensor = logits[0, -1:].argmax(dim=-1)
        idx = torch.cat([idx, token_idx.unsqueeze(0)], dim=1)

        if token_idx.item() == dataset.end_idx:
            break
        else:
            gen_string = gen_string + dataset.idx_to_token[int(token_idx.
 ↪item())]
    return gen_string


def encdec_translate(
    sentence: str, model: TransformerEncoderDecoder, dataset:␣
 ↪EncoderDecoderDataset
):
    words = sentence.split()
    return " ".join([encdec_generate(w, model, dataset) for w in words])
```

[19]:
```
encdec_translate("the", model, encdec_ds_small)
```

[19]:
```
'ethay'
```

[20]:
```
encdec_translate("the air conditioning is working", model, encdec_ds_small)
```

[20]:
```
'ethay airway onditioningcay isway orkingway'
```

[21]:
```
encdec_translate("testing some random words", model, encdec_ds_small)
```

[21]:
```
'estingtay omesay andomray ordsway'
```

[ ]:
```
configs = [
    (16, "small", encdec_ds_small),
    (16, "large", encdec_ds_large),
    (32, "small", encdec_ds_small),
    (32, "large", encdec_ds_large),
]

all_results = {}

for hidden_size, dataset_name, dataset in configs:
    print(f"\nTraining model: hidden_size={hidden_size},␣
 ↪dataset={dataset_name}")

    # model = load_model_auto(
```

```
    #     TransformerEncoderDecoder,
    #     vocab_size=len(dataset.idx_to_token),
    #     hidden_size=hidden_size,
    #     num_layers=4,
    #     dataset_name=dataset_name,
    # )
    model = TransformerEncoderDecoder(
        vocab_size=len(dataset.idx_to_token),
        hidden_size=hidden_size,
        num_layers=4,
        dataset_name=dataset_name,
    )

    optimizer = optim.AdamW(model.parameters(), lr=1e-2)
    scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)

    results = train(
        model=model,
        dataset=dataset,
        num_epochs=NUM_EPOCHS,
        optimizer=optimizer,
        scheduler=scheduler,
        device=device,
        batch_size=32 if dataset_name == "small" else 256,
    )
    all_results[(hidden_size, dataset_name)] = results
    save_model_auto(model)

save_loss_comparison_by_dataset(all_results)
save_loss_comparison_by_hidden(all_results)
```

```
Training model: hidden_size=16, dataset=small
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/TransformerEncoderDecoder/Model_ds_namesmall_vocab30_hid1
6_layers4.pt onto mps device.
Epoch 1 | Train loss: 0.282690 | Val loss 0.119487
Epoch 2 | Train loss: 0.100203 | Val loss 0.125501
Epoch 3 | Train loss: 0.096975 | Val loss 0.092090
Epoch 4 | Train loss: 0.078260 | Val loss 0.084708
Epoch 5 | Train loss: 0.072440 | Val loss 0.088962
Epoch 6 | Train loss: 0.066571 | Val loss 0.072962
Epoch 7 | Train loss: 0.065810 | Val loss 0.065583
Epoch 8 | Train loss: 0.054039 | Val loss 0.067781
Epoch 9 | Train loss: 0.052774 | Val loss 0.079471
Epoch 10 | Train loss: 0.061673 | Val loss 0.069800
Epoch 11 | Train loss: 0.046289 | Val loss 0.075188
Epoch 12 | Train loss: 0.043467 | Val loss 0.048739
```

```
Epoch 13 | Train loss: 0.036280 | Val loss 0.044235
Epoch 14 | Train loss: 0.030028 | Val loss 0.040429
Epoch 15 | Train loss: 0.024368 | Val loss 0.045205
Epoch 16 | Train loss: 0.022556 | Val loss 0.037809
Epoch 17 | Train loss: 0.023777 | Val loss 0.061744
Epoch 18 | Train loss: 0.025582 | Val loss 0.042941
Epoch 19 | Train loss: 0.023655 | Val loss 0.048473
Epoch 20 | Train loss: 0.024986 | Val loss 0.055363
Epoch 21 | Train loss: 0.022103 | Val loss 0.048486
Epoch 22 | Train loss: 0.026233 | Val loss 0.045333
Epoch 23 | Train loss: 0.020105 | Val loss 0.044494
Epoch 24 | Train loss: 0.017601 | Val loss 0.048049
Epoch 25 | Train loss: 0.015755 | Val loss 0.035441

Training model: hidden_size=16, dataset=large
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/TransformerEncoderDecoder/Model_ds_namelarge_vocab30_hid1
6_layers4.pt onto mps device.
Epoch 1 | Train loss: 1.752035 | Val loss 1.451140
Epoch 2 | Train loss: 1.180100 | Val loss 0.924659
Epoch 3 | Train loss: 0.687570 | Val loss 0.501195
Epoch 4 | Train loss: 0.324483 | Val loss 0.233330
Epoch 5 | Train loss: 0.159920 | Val loss 0.129188
Epoch 6 | Train loss: 0.105429 | Val loss 0.109710
Epoch 7 | Train loss: 0.079490 | Val loss 0.076954
Epoch 8 | Train loss: 0.059346 | Val loss 0.057958
Epoch 9 | Train loss: 0.051220 | Val loss 0.048695
Epoch 10 | Train loss: 0.046706 | Val loss 0.051182
Epoch 11 | Train loss: 0.040899 | Val loss 0.049593
Epoch 12 | Train loss: 0.045514 | Val loss 0.048623
Epoch 13 | Train loss: 0.033743 | Val loss 0.046426
Epoch 14 | Train loss: 0.031032 | Val loss 0.038175
Epoch 15 | Train loss: 0.025308 | Val loss 0.037642
Epoch 16 | Train loss: 0.024815 | Val loss 0.055294
Epoch 17 | Train loss: 0.033792 | Val loss 0.036898
Epoch 18 | Train loss: 0.022894 | Val loss 0.029995
Epoch 19 | Train loss: 0.020354 | Val loss 0.029650
Epoch 20 | Train loss: 0.024549 | Val loss 0.037606
Epoch 21 | Train loss: 0.018849 | Val loss 0.029841
Epoch 22 | Train loss: 0.022382 | Val loss 0.054181
Epoch 23 | Train loss: 0.021724 | Val loss 0.028102
Epoch 24 | Train loss: 0.018688 | Val loss 0.031005
Epoch 25 | Train loss: 0.016014 | Val loss 0.029370

Training model: hidden_size=32, dataset=small
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/TransformerEncoderDecoder/Model_ds_namesmall_vocab30_hid3
2_layers4.pt onto mps device.
```

```
Epoch 1 | Train loss: 1.908707 | Val loss 1.791376
Epoch 2 | Train loss: 1.339528 | Val loss 1.073616
Epoch 3 | Train loss: 0.691916 | Val loss 0.417469
Epoch 4 | Train loss: 0.329485 | Val loss 0.226425
Epoch 5 | Train loss: 0.215454 | Val loss 0.190468
Epoch 6 | Train loss: 0.166157 | Val loss 0.140377
Epoch 7 | Train loss: 0.130177 | Val loss 0.110813
Epoch 8 | Train loss: 0.093311 | Val loss 0.092226
Epoch 9 | Train loss: 0.088159 | Val loss 0.123600
Epoch 10 | Train loss: 0.088821 | Val loss 0.061677
Epoch 11 | Train loss: 0.061109 | Val loss 0.071283
Epoch 12 | Train loss: 0.047845 | Val loss 0.063630
Epoch 13 | Train loss: 0.046148 | Val loss 0.052610
Epoch 14 | Train loss: 0.042500 | Val loss 0.066143
Epoch 15 | Train loss: 0.040155 | Val loss 0.044555
Epoch 16 | Train loss: 0.032502 | Val loss 0.050437
Epoch 17 | Train loss: 0.026377 | Val loss 0.038128
Epoch 18 | Train loss: 0.025233 | Val loss 0.052488
Epoch 19 | Train loss: 0.027906 | Val loss 0.036548
Epoch 20 | Train loss: 0.016713 | Val loss 0.039828
Epoch 21 | Train loss: 0.018074 | Val loss 0.038401
Epoch 22 | Train loss: 0.014226 | Val loss 0.032558
Epoch 23 | Train loss: 0.018394 | Val loss 0.037174
Epoch 24 | Train loss: 0.017545 | Val loss 0.034607
Epoch 25 | Train loss: 0.013835 | Val loss 0.032371

Training model: hidden_size=32, dataset=large
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/TransformerEncoderDecoder/Model_ds_namelarge_vocab30_hid3
2_layers4.pt onto mps device.
Epoch 1 | Train loss: 1.869922 | Val loss 1.322680
Epoch 2 | Train loss: 0.861228 | Val loss 0.365645
Epoch 3 | Train loss: 0.294439 | Val loss 0.231007
Epoch 4 | Train loss: 0.147985 | Val loss 0.088157
Epoch 5 | Train loss: 0.111792 | Val loss 0.125666
Epoch 6 | Train loss: 0.053795 | Val loss 0.049737
Epoch 7 | Train loss: 0.040531 | Val loss 0.108465
Epoch 8 | Train loss: 0.039519 | Val loss 0.047647
Epoch 9 | Train loss: 0.030697 | Val loss 0.033957
Epoch 10 | Train loss: 0.021484 | Val loss 0.069264
Epoch 11 | Train loss: 0.025629 | Val loss 0.032973
Epoch 12 | Train loss: 0.017451 | Val loss 0.022390
Epoch 13 | Train loss: 0.014279 | Val loss 0.022728
Epoch 14 | Train loss: 0.011467 | Val loss 0.018021
Epoch 15 | Train loss: 0.012247 | Val loss 0.038864
Epoch 16 | Train loss: 0.011687 | Val loss 0.016182
Epoch 17 | Train loss: 0.008074 | Val loss 0.015719
Epoch 18 | Train loss: 0.007249 | Val loss 0.021653
```

```
Epoch 19 | Train loss: 0.007246 | Val loss 0.020273
Epoch 20 | Train loss: 0.007894 | Val loss 0.018705
Epoch 21 | Train loss: 0.006733 | Val loss 0.015223
Epoch 22 | Train loss: 0.009007 | Val loss 0.030844
Epoch 23 | Train loss: 0.008515 | Val loss 0.026953
Epoch 24 | Train loss: 0.011375 | Val loss 0.016680
Epoch 25 | Train loss: 0.006743 | Val loss 0.019985
```

```
<Figure size 640x480 with 0 Axes>
```

```
<Figure size 640x480 with 0 Axes>
```

2 Part 2: Decoder-only Transformers and Multi-Head Attention

In this section, you'll train transformers that only use a decoder instead of a dual encoder-decoder architecture. You'll also implement and experiment with multi-head attention. Finally, you'll compare increasing parameter count with multi-head attention vs. increasing the parameter count with a larger hidden dimensionality.

2.1 Step 1: Implement the Decoder-Only Forward Method

[23]:
```python
class DecoderOnlyBlock(nn.Module):
    def __init__(self, hidden_size: int):
        super().__init__()
        self.norm1 = LayerNorm(hidden_size)
        self.norm2 = LayerNorm(hidden_size)
        self.attn = CausalAttention(hidden_size)
        self.mlp = MLP(hidden_size)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor):
        context, attention = self.attn(self.norm1(x))
        x = x + context
        x = x + self.mlp(self.norm2(x))
        return x, attention


class DecoderTransformer(nn.Module):
    def __init__(self, vocab_size: int, hidden_size: int, num_layers: int):
        super().__init__()

        self.tok_emb = nn.Embedding(vocab_size, hidden_size)
        self.pos_emb = PositionalEncoding(hidden_size)

        self.blocks = nn.ModuleList(
            [DecoderOnlyBlock(hidden_size) for i in range(num_layers)]
        )
```

```
        self.head = nn.Linear(hidden_size, vocab_size, bias=False)
        self.head.weight = self.tok_emb.weight
        # weight tie, the final layer reuses the weights of the embedding layers
        # this reduces the total number of parameters
        self.to(device)
        self.filename = model_filename(
            hidden_size=hidden_size, num_layers=num_layers,␣
↪vocab_size=vocab_size
        )

    def forward(self, inputs: torch.Tensor):
        """Forward pass of the Transformer decoder.

        Arguments:
            inputs: Input token indexes across a batch for all the time step.␣
↪(batch_size x decoder_seq_len)
        Returns:
            output: Un-normalized scores for each token in the vocabulary,␣
↪across a batch for all the decoding time steps. (batch_size x␣
↪decoder_seq_len x vocab_size)
            attentions: The stacked attention weights applied to the encoder␣
↪annotations (batch_size x encoder_seq_len x decoder_seq_len)
        """
        # ------------
        # FILL THIS IN
        # ------------
        # pos =
        # x =
        # self_attn = []

        # for block in self.blocks:
        #     ...
        #
        # unormalized_scores =
        pos = torch.arange(0, inputs.shape[-1], device=inputs.device).long()
        x = self.tok_emb(inputs) + self.pos_emb(pos)
        self_attn = []
        for block in self.blocks:
            x, s = block(x)
            self_attn.append(s)
        unormalized_scores = self.head(x)

        return unormalized_scores, {"self_attn": self_attn}
```

[24]:
```
decoder_ds_small = DecoderDataset(data_path, "pig_latin_small")
decoder_ds_large = DecoderDataset(data_path, "pig_latin_large")
```

[25]:
```
print("Dataset sample:")
print("Input:", decoder_ds_small.to_token(decoder_ds_small[0][0].tolist()))
print("Target:", decoder_ds_small.to_token(decoder_ds_small[0][1].tolist()))
```

```
Dataset sample:
Input: <SOS>reconcile<EOP>econcileray
Target: reconcile<EOP>econcileray<EOS>
```

2.2 Step 2: Format for decoder-only generation.

Implement the missing line in `decoder_generate` to create an input string that looks like:

`<SOS> input sequence <EOP>`

You should use the `dataset.start_idx`, `dataset.eop_idx` attributes and the `dataset.to_idx()` method.

[26]:
```
def decoder_generate(
    input: str,
    model: DecoderTransformer,
    dataset: DecoderDataset,
    max_len: int = 20,
):
    gen_string = ""
    # idx = TODO
    idx = dataset.to_idx(input)
    idx = torch.tensor([idx]).to(device)

    for i in range(max_len):
        logits = model(idx)[0]

        token_idx = logits[0, -1:].argmax(dim=-1)
        idx = torch.cat([idx, token_idx.unsqueeze(0)], dim=1)

        if token_idx.item() == dataset.end_idx:
            break
        else:
            gen_string = gen_string + dataset.idx_to_token[token_idx.item()]
    return gen_string


def decoder_translate(
    sentence: str, model: DecoderTransformer, dataset: DecoderDataset
):
    words = sentence.split()
    return " ".join([decoder_generate(w, model, dataset) for w in words])
```

2.3 Step 3: Train the model

We'll now train a decoder only model and test it out on an example. Use the output of this to guide your answer for question 3.

[ ]:
```
# model = load_model_auto(
#     DecoderTransformer,
#     vocab_size=len(decoder_ds_small.idx_to_token),
#     hidden_size=16,
#     num_layers=4,
# )
model = DecoderTransformer(
    vocab_size=len(decoder_ds_small.idx_to_token), hidden_size=16, num_layers=4
)
```

```
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/DecoderTransformer/Model_vocab30_hid16_layers4.pt onto
mps device.
```

[28]:
```
optimizer = optim.AdamW(model.parameters(), lr=1e-2)
scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)

results = train(
    model=model,
    dataset=decoder_ds_small,
    num_epochs=NUM_EPOCHS,
    optimizer=optimizer,
    scheduler=scheduler,
    device=device,
    batch_size=32,
)
save_model_auto(model)
```

```
Epoch 1 | Train loss: 1.750797 | Val loss 1.543421
Epoch 2 | Train loss: 1.380129 | Val loss 1.224484
Epoch 3 | Train loss: 1.060866 | Val loss 0.986937
Epoch 4 | Train loss: 0.831978 | Val loss 0.764885
Epoch 5 | Train loss: 0.670749 | Val loss 0.790287
Epoch 6 | Train loss: 0.563655 | Val loss 0.504474
Epoch 7 | Train loss: 0.469036 | Val loss 0.553065
Epoch 8 | Train loss: 0.384653 | Val loss 0.436691
Epoch 9 | Train loss: 0.340577 | Val loss 0.286533
Epoch 10 | Train loss: 0.291354 | Val loss 0.356906
Epoch 11 | Train loss: 0.275355 | Val loss 0.218699
Epoch 12 | Train loss: 0.235852 | Val loss 0.234852
Epoch 13 | Train loss: 0.207349 | Val loss 0.194345
Epoch 14 | Train loss: 0.202178 | Val loss 0.226432
Epoch 15 | Train loss: 0.181114 | Val loss 0.174726
Epoch 16 | Train loss: 0.160138 | Val loss 0.167639
```

```
Epoch 17 | Train loss: 0.151384 | Val loss 0.125920
Epoch 18 | Train loss: 0.141745 | Val loss 0.153938
Epoch 19 | Train loss: 0.149035 | Val loss 0.134451
Epoch 20 | Train loss: 0.124795 | Val loss 0.127090
Epoch 21 | Train loss: 0.104896 | Val loss 0.123745
Epoch 22 | Train loss: 0.104194 | Val loss 0.109657
Epoch 23 | Train loss: 0.100252 | Val loss 0.142833
Epoch 24 | Train loss: 0.098598 | Val loss 0.119624
Epoch 25 | Train loss: 0.091952 | Val loss 0.105700
```

[28]:
```
PosixPath('/Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/DecoderTransformer/Model_vocab30_hid16_layers4.pt')
```

[29]:
```
decoder_translate("the air conditioning is working", model, decoder_ds_small)
```

[29]:
```
'fohefohgggggggggggg gggrgrgggggggggggggg bondccioniningondunr
sssssssssssssssssss rlh'
```

2.4 Step 4: Implement Multi-Head Attention

In this next step, you'll implement multi-head attention. In this setting, we'll have the dimensionality for each of the heads be (total hidden dimensionality)/(number of heads). We'll use a projection head to combine the outputs of these different attention heads after concatenating their outputs together (which can also be achieved using tensor reshaping).

[30]:
```
# TO DO:


def multihead_scaled_dot_attention(
    q: torch.Tensor, k: torch.Tensor, v: torch.Tensor, mask: torch.Tensor |␣
 ↪None = None
):
    """Compute multi-head dot product attention.
      q : shape (batch, num_heads, k, hidden_size)
      k : shape (batch, num_heads, seq_len, hidden_size)
      v : shape (batch, num_heads, seq_len, hidden_size)

    returns:
        context   : shape (batch, num_heads, k, hidden_size)
        attention : shape (batch, num_heads, k, seq_len)
    """
    # Hint: if you implemented scaled_dot_attention well, the code for this␣
 ↪should be the exact same.

    # ------------
    # FILL THIS IN
    # ------------
    # unnorm_attn =
```

```
    # masked_unnorm_attn =
    # attention =
    # context =
    unnorm_attn = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(q.size(-1))
    if mask is not None:
        unnorm_attn = unnorm_attn.masked_fill(mask == 0, float("-inf"))
    attention = F.softmax(unnorm_attn, dim=-1)
    context = torch.matmul(attention, v)
    return context, attention
```

[31]:
```
class MultiheadCausalAttention(Attention):
    proj: nn.Linear

    def __init__(self, hidden_size: int, num_heads: int):
        super().__init__(hidden_size)
        # self.Q, self.K, self.V already defined as nn.Linear(hidden_size,␣
↪hidden_size)
        self.num_heads = num_heads
        # ------------
        # FILL THIS IN
        # ------------
        # if self.num_heads > 1:
        #   self.proj = nn.Linear( ,  , bias=False)
        if self.num_heads > 1:
            self.proj = nn.Linear(hidden_size, hidden_size, bias=False)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size,␣
↪num_heads=num_heads)

    def forward(self, x: torch.Tensor):
        """Compute causal multi-head dot product attention from an input␣
↪sequence.
          x : shape ( batch, seq_len, hidden_size)
          k : shape (batch, num_heads, seq_len, hidden_size)
          v : shape (batch, num_heads, seq_len, hidden_size)

        returns:
            context : shape ( batch, seq_len, hidden_size)
            attention : shape (batch, num_heads, k, seq_len)
        """
        # ------------
        # FILL THIS IN
        # ------------
        # unnorm_attn =
        # masked_unnorm_attn =
        # attention =
        # context =
```

```
        # B, T, C = x.size()
        # q = self.Q(x).view( ,  ,  ,  ).transpose(1, 2)
        # k = self.K(x).view( ,  ,  ,  ).transpose(1, 2)
        # v = self.V(x).view( ,  ,  ,  ).transpose(1, 2)

        # mask =

        # context, attention = multihead_scaled_dot_attention(q, k, v, mask)

        # Hint: you may find the .transpose(), .contiguous(), and .view()
        # methods helpful for creating the reshaped context.

        # reshaped_context =

        # Only perform a projection if the number of heads is more than 1.
        B, T, C = x.size()
        q = self.Q(x).view(B, T, self.num_heads, C // self.num_heads).
↪transpose(1, 2)
        k = self.K(x).view(B, T, self.num_heads, C // self.num_heads).
↪transpose(1, 2)
        v = self.V(x).view(B, T, self.num_heads, C // self.num_heads).
↪transpose(1, 2)
        mask = torch.tril(torch.ones((T, T), device=x.device)).bool()
        context, attention = multihead_scaled_dot_attention(q, k, v, mask)
        reshaped_context = context.transpose(1, 2).contiguous().view(B, T, C)
        if self.num_heads > 1:

            return self.proj(reshaped_context), attention

        else:

            return reshaped_context, attention
```

[32]:
```
class MultiheadDecoderBlock(DecoderOnlyBlock):
    def __init__(self, hidden_size: int, num_heads: int):
        super().__init__(hidden_size)
        self.attn = MultiheadCausalAttention(hidden_size, num_heads)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size,␣
↪num_heads=num_heads)


class MultiheadDecoderTransformer(DecoderTransformer):
    def __init__(
        self, vocab_size: int, hidden_size: int, num_layers: int, num_heads: int
    ):
```

```
        super().__init__(vocab_size, hidden_size, num_layers)
        self.num_heads = num_heads
        self.num_layers = num_layers
        self.vocab_size = vocab_size
        self.hidden_size = hidden_size

        self.blocks = nn.ModuleList(
            [MultiheadDecoderBlock(hidden_size, num_heads) for i in␣
↪range(num_layers)]
        )
        self.to(device)
        self.filename = model_filename(
            hidden_size=hidden_size,
            num_layers=num_layers,
            vocab_size=vocab_size,
            num_heads=num_heads,
        )
```

2.5 Step 5: Compare Parameter Usage Methods

Here, you'll train 5 models with varying numbers of attention heads and hidden dimensionalities. You'll compare their parameter efficiency. Start by implementing the `calc_parameters_decoder` method, and then run the code to train the models.

[33]:
```
def calc_parameters_decoder(model: MultiheadDecoderTransformer) -> int:
    # ------------
    # FILL THIS IN
    # ------------
    # params = TODO # embedding
    # params +=  TODO # QKV linear projections
    # if model.num_heads > 1:
    #   params += TODO # attention linear projection
    # params += TODO # MLP linear
    # params += TODO # layer norms Hint: LayerNorm(x) uses x parameters

    # Hint: Read the code carefully:
    # params += TODO # head
    params = model.vocab_size * model.hidden_size  # embedding
    params += 3 * (model.hidden_size * model.hidden_size)  # QKV linear␣
↪projections
    if model.num_heads > 1:
        params += model.hidden_size * model.hidden_size  # attention linear␣
↪projection
    params += 2 * (
        4 * model.hidden_size * model.hidden_size
    )  # MLP linear (2 per block)
    params += 2 * 2 * model.hidden_size  # layer norms (2 per block)
```

```
    return params
```

[ ]:
```
import matplotlib.pyplot as plt
import numpy as np

configs = [
    (16, 4, 4, 64),  # (hidden_size, num_heads, num_layers, batch_size)
    (16, 2, 4, 64),
    (16, 1, 4, 64),
    (24, 1, 4, 64),
    (32, 1, 4, 64),
]

all_results = {}

for hidden_size, num_heads, num_layers, batch_size in configs:
    print(
        f"\nTraining model: hidden_size={hidden_size}, num_heads={num_heads},␣
 ↪num_layers={num_layers}"
    )
    # model = load_model_auto(
    #     MultiheadDecoderTransformer,
    #     vocab_size=len(decoder_ds_large.idx_to_token),
    #     hidden_size=hidden_size,
    #     num_layers=num_layers,
    #     num_heads=num_heads,
    # )
    model = MultiheadDecoderTransformer(
        vocab_size=len(decoder_ds_large.idx_to_token),
        hidden_size=hidden_size,
        num_layers=num_layers,
        num_heads=num_heads,
    )

    optimizer = optim.AdamW(model.parameters(), lr=5e-3)
    scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)

    results = train(
        model=model,
        dataset=decoder_ds_large,
        num_epochs=NUM_EPOCHS,
        optimizer=optimizer,
        scheduler=scheduler,
        device=device,
        batch_size=batch_size,
    )
```

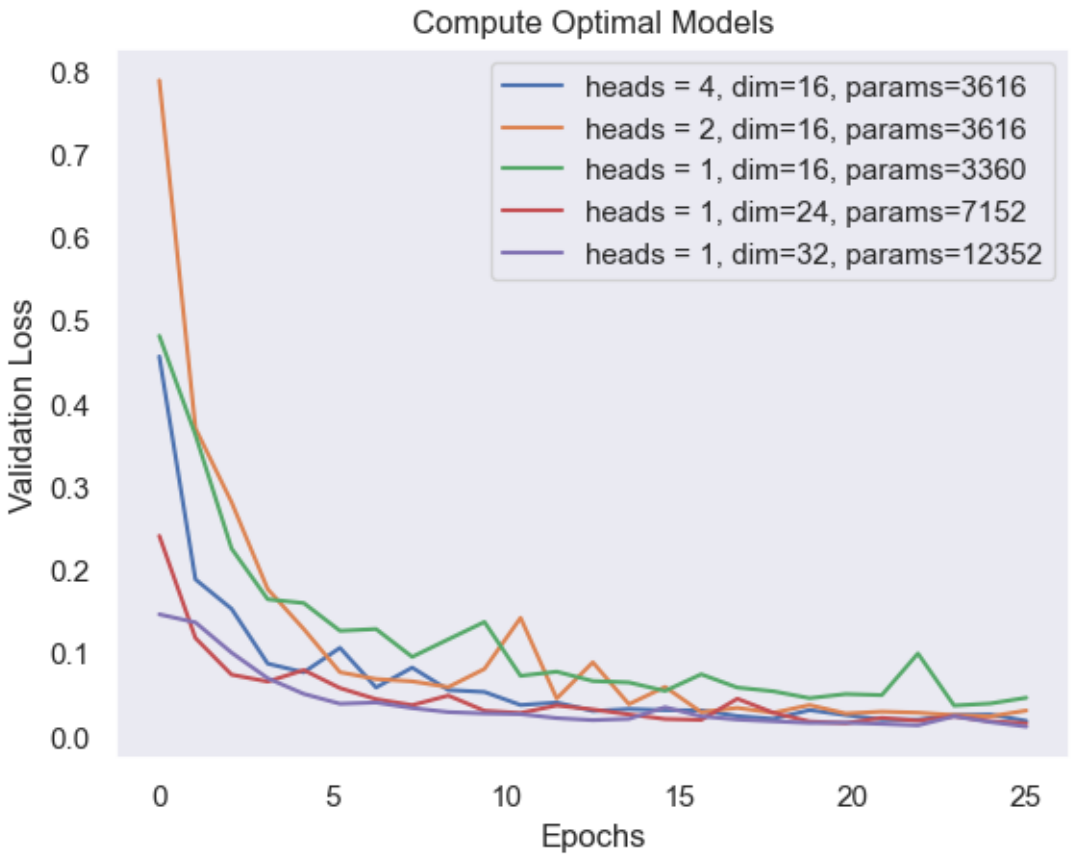
```
    save_model_auto(model)

    parameters = calc_parameters_decoder(model)
    x_points = np.linspace(0, len(results["val_loss"]),␣
 ↪len(results["val_loss"]))
    plt.plot(
        x_points,
        results["val_loss"],
        label="heads = {}, dim={}, params={}".format(
            num_heads, hidden_size, parameters
        ),
    )

plt.xlabel("Epochs")
plt.ylabel("Validation Loss")
plt.title("Compute Optimal Models")
plt.grid()
plt.legend()
plt.show()
```

```
Training model: hidden_size=16, num_heads=4, num_layers=4
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid16_layers4_h
eads4.pt onto mps device.
Epoch 1 | Train loss: 0.813669 | Val loss 0.456453
Epoch 2 | Train loss: 0.318358 | Val loss 0.188327
Epoch 3 | Train loss: 0.181642 | Val loss 0.152886
Epoch 4 | Train loss: 0.124241 | Val loss 0.086995
Epoch 5 | Train loss: 0.096061 | Val loss 0.076589
Epoch 6 | Train loss: 0.085140 | Val loss 0.106006
Epoch 7 | Train loss: 0.074020 | Val loss 0.058294
Epoch 8 | Train loss: 0.060551 | Val loss 0.082235
Epoch 9 | Train loss: 0.053440 | Val loss 0.055254
Epoch 10 | Train loss: 0.046854 | Val loss 0.052933
Epoch 11 | Train loss: 0.039619 | Val loss 0.037227
Epoch 12 | Train loss: 0.036588 | Val loss 0.040144
Epoch 13 | Train loss: 0.031300 | Val loss 0.030104
Epoch 14 | Train loss: 0.029731 | Val loss 0.032388
Epoch 15 | Train loss: 0.025695 | Val loss 0.031068
Epoch 16 | Train loss: 0.025639 | Val loss 0.030745
Epoch 17 | Train loss: 0.023204 | Val loss 0.023949
Epoch 18 | Train loss: 0.018776 | Val loss 0.020600
Epoch 19 | Train loss: 0.020193 | Val loss 0.031106
Epoch 20 | Train loss: 0.018875 | Val loss 0.025123
Epoch 21 | Train loss: 0.016650 | Val loss 0.019878
Epoch 22 | Train loss: 0.015947 | Val loss 0.019585
Epoch 23 | Train loss: 0.016648 | Val loss 0.025075
```

```
Epoch 24 | Train loss: 0.013225 | Val loss 0.025776
Epoch 25 | Train loss: 0.013089 | Val loss 0.018158

Training model: hidden_size=16, num_heads=2, num_layers=4
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid16_layers4_h
eads2.pt onto mps device.
Epoch 1 | Train loss: 1.018973 | Val loss 0.788565
Epoch 2 | Train loss: 0.500485 | Val loss 0.370172
Epoch 3 | Train loss: 0.265157 | Val loss 0.281128
Epoch 4 | Train loss: 0.187128 | Val loss 0.176626
Epoch 5 | Train loss: 0.143767 | Val loss 0.128517
Epoch 6 | Train loss: 0.114133 | Val loss 0.076675
Epoch 7 | Train loss: 0.095616 | Val loss 0.068469
Epoch 8 | Train loss: 0.082454 | Val loss 0.065698
Epoch 9 | Train loss: 0.079078 | Val loss 0.058812
Epoch 10 | Train loss: 0.061667 | Val loss 0.080626
Epoch 11 | Train loss: 0.059266 | Val loss 0.142069
Epoch 12 | Train loss: 0.050418 | Val loss 0.045068
Epoch 13 | Train loss: 0.048671 | Val loss 0.088596
Epoch 14 | Train loss: 0.037041 | Val loss 0.038197
Epoch 15 | Train loss: 0.040018 | Val loss 0.058965
Epoch 16 | Train loss: 0.031405 | Val loss 0.028674
Epoch 17 | Train loss: 0.028455 | Val loss 0.033782
Epoch 18 | Train loss: 0.029210 | Val loss 0.027892
Epoch 19 | Train loss: 0.024781 | Val loss 0.037200
Epoch 20 | Train loss: 0.025257 | Val loss 0.027041
Epoch 21 | Train loss: 0.023607 | Val loss 0.029257
Epoch 22 | Train loss: 0.020624 | Val loss 0.028002
Epoch 23 | Train loss: 0.020528 | Val loss 0.025124
Epoch 24 | Train loss: 0.018870 | Val loss 0.023551
Epoch 25 | Train loss: 0.018044 | Val loss 0.030579

Training model: hidden_size=16, num_heads=1, num_layers=4
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid16_layers4_h
eads1.pt onto mps device.
Epoch 1 | Train loss: 0.745471 | Val loss 0.481261
Epoch 2 | Train loss: 0.438703 | Val loss 0.362150
Epoch 3 | Train loss: 0.301155 | Val loss 0.225376
Epoch 4 | Train loss: 0.224585 | Val loss 0.164387
Epoch 5 | Train loss: 0.193241 | Val loss 0.159801
Epoch 6 | Train loss: 0.156916 | Val loss 0.126491
Epoch 7 | Train loss: 0.138336 | Val loss 0.128567
Epoch 8 | Train loss: 0.118482 | Val loss 0.095112
Epoch 9 | Train loss: 0.115696 | Val loss 0.116222
Epoch 10 | Train loss: 0.100234 | Val loss 0.137132
Epoch 11 | Train loss: 0.091130 | Val loss 0.072352
```

```
Epoch 12 | Train loss: 0.078085 | Val loss 0.077328
Epoch 13 | Train loss: 0.081894 | Val loss 0.066050
Epoch 14 | Train loss: 0.063385 | Val loss 0.064379
Epoch 15 | Train loss: 0.066233 | Val loss 0.054153
Epoch 16 | Train loss: 0.064165 | Val loss 0.074182
Epoch 17 | Train loss: 0.054361 | Val loss 0.058246
Epoch 18 | Train loss: 0.051486 | Val loss 0.053706
Epoch 19 | Train loss: 0.047319 | Val loss 0.045460
Epoch 20 | Train loss: 0.047748 | Val loss 0.050424
Epoch 21 | Train loss: 0.043327 | Val loss 0.049133
Epoch 22 | Train loss: 0.038748 | Val loss 0.099252
Epoch 23 | Train loss: 0.037680 | Val loss 0.036589
Epoch 24 | Train loss: 0.036105 | Val loss 0.039024
Epoch 25 | Train loss: 0.035018 | Val loss 0.045957

Training model: hidden_size=24, num_heads=1, num_layers=4
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid24_layers4_h
eads1.pt onto mps device.
Epoch 1 | Train loss: 0.382138 | Val loss 0.240671
Epoch 2 | Train loss: 0.148967 | Val loss 0.117768
Epoch 3 | Train loss: 0.095751 | Val loss 0.073707
Epoch 4 | Train loss: 0.076854 | Val loss 0.065455
Epoch 5 | Train loss: 0.065960 | Val loss 0.079231
Epoch 6 | Train loss: 0.050421 | Val loss 0.057626
Epoch 7 | Train loss: 0.044531 | Val loss 0.044251
Epoch 8 | Train loss: 0.036336 | Val loss 0.037041
Epoch 9 | Train loss: 0.033996 | Val loss 0.048603
Epoch 10 | Train loss: 0.029491 | Val loss 0.030315
Epoch 11 | Train loss: 0.027907 | Val loss 0.027635
Epoch 12 | Train loss: 0.023680 | Val loss 0.036719
Epoch 13 | Train loss: 0.024870 | Val loss 0.031910
Epoch 14 | Train loss: 0.018978 | Val loss 0.025906
Epoch 15 | Train loss: 0.020195 | Val loss 0.020388
Epoch 16 | Train loss: 0.018747 | Val loss 0.019331
Epoch 17 | Train loss: 0.015067 | Val loss 0.044780
Epoch 18 | Train loss: 0.015846 | Val loss 0.027907
Epoch 19 | Train loss: 0.013938 | Val loss 0.017379
Epoch 20 | Train loss: 0.013059 | Val loss 0.015499
Epoch 21 | Train loss: 0.011967 | Val loss 0.021743
Epoch 22 | Train loss: 0.011263 | Val loss 0.018424
Epoch 23 | Train loss: 0.010913 | Val loss 0.023736
Epoch 24 | Train loss: 0.010483 | Val loss 0.017777
Epoch 25 | Train loss: 0.009405 | Val loss 0.014990

Training model: hidden_size=32, num_heads=1, num_layers=4
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid32_layers4_h
```

```
eads1.pt onto mps device.
Epoch 1 | Train loss: 0.286629 | Val loss 0.146302
Epoch 2 | Train loss: 0.125485 | Val loss 0.136549
Epoch 3 | Train loss: 0.095803 | Val loss 0.100186
Epoch 4 | Train loss: 0.069927 | Val loss 0.069415
Epoch 5 | Train loss: 0.059556 | Val loss 0.050596
Epoch 6 | Train loss: 0.043010 | Val loss 0.038776
Epoch 7 | Train loss: 0.035310 | Val loss 0.040229
Epoch 8 | Train loss: 0.030662 | Val loss 0.033544
Epoch 9 | Train loss: 0.026778 | Val loss 0.028711
Epoch 10 | Train loss: 0.025704 | Val loss 0.027127
Epoch 11 | Train loss: 0.023963 | Val loss 0.026332
Epoch 12 | Train loss: 0.018128 | Val loss 0.021484
Epoch 13 | Train loss: 0.017573 | Val loss 0.019068
Epoch 14 | Train loss: 0.015893 | Val loss 0.020552
Epoch 15 | Train loss: 0.015352 | Val loss 0.034526
Epoch 16 | Train loss: 0.012132 | Val loss 0.023576
Epoch 17 | Train loss: 0.011141 | Val loss 0.019705
Epoch 18 | Train loss: 0.012571 | Val loss 0.017661
Epoch 19 | Train loss: 0.009691 | Val loss 0.015707
Epoch 20 | Train loss: 0.007921 | Val loss 0.015915
Epoch 21 | Train loss: 0.008482 | Val loss 0.014529
Epoch 22 | Train loss: 0.007229 | Val loss 0.012837
Epoch 23 | Train loss: 0.008125 | Val loss 0.023578
Epoch 24 | Train loss: 0.007693 | Val loss 0.016775
Epoch 25 | Train loss: 0.003847 | Val loss 0.011257
```



[35]:
```
plt.clf()
```

```
<Figure size 640x480 with 0 Axes>
```

3 Part 3: IsoFLOP Profiling and Scaling Laws

In this final section, you'll compare different architectures based on their compute efficiency. You will also determine the optimal number of parameters and tokens for a fixed compute budget.

While this section does not require any additional code from you, **use the output of it to guide your written responses**.

[ ]:
```
from utils import (
    plot_scaling_law,
    plot_scaling_law_poly,
    interpolate,
    plot_flops_tokens,
    plot_isoflop,
    plot_flops_params,
)
```

```
import numpy as np
```

3.1 Step 1: Train models, Compare FLOPS vs Validation Loss

We'll inspect the compute efficiency of different models by plotting their validation loss against the number of FLOPs used in training up to that point.

[37]:
```
from types_hw2 import TrainerHistDict
```

[ ]:
```
configs = [
    (40, 2, 4, 256),  # (hidden_size, num_heads, num_layers, batch_size)
    (32, 2, 4, 128),
    (24, 2, 4, 64),
    (16, 1, 4, 64),
    (16, 1, 3, 64),
    (12, 1, 3, 64),
    (12, 1, 2, 64),
]

all_results: dict[int, TrainerHistDict] = {}

for hidden_size, num_heads, num_layers, batch_size in configs:
    print(
        f"\nTraining model: hidden_size={hidden_size}, num_heads={num_heads},␣
 ↪num_layers={num_layers}"
    )

    # model = load_model_auto(
    #     MultiheadDecoderTransformer,
    #     vocab_size=len(decoder_ds_large.idx_to_token),
    #     hidden_size=hidden_size,
    #     num_layers=num_layers,
    #     num_heads=num_heads,
    # )
    model = MultiheadDecoderTransformer(
        vocab_size=len(decoder_ds_large.idx_to_token),
        hidden_size=hidden_size,
        num_layers=num_layers,
        num_heads=num_heads,
    )

    optimizer = optim.AdamW(model.parameters(), lr=5e-3)
    scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)

    results = train(
        model=model,
```

```
        dataset=decoder_ds_large,
        num_epochs=NUM_EPOCHS,
        optimizer=optimizer,
        scheduler=scheduler,
        device=device,
        batch_size=batch_size,
    )
    save_model_auto(model)

    parameters = calc_parameters_decoder(model)
    all_results[parameters] = results
```

```
Training model: hidden_size=40, num_heads=2, num_layers=4
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid40_layers4_h
eads2.pt onto mps device.
Epoch 1 | Train loss: 1.528885 | Val loss 0.670217
Epoch 2 | Train loss: 0.346934 | Val loss 0.153018
Epoch 3 | Train loss: 0.123261 | Val loss 0.108073
Epoch 4 | Train loss: 0.062645 | Val loss 0.078309
Epoch 5 | Train loss: 0.047663 | Val loss 0.045472
Epoch 6 | Train loss: 0.031922 | Val loss 0.052582
Epoch 7 | Train loss: 0.033140 | Val loss 0.030728
Epoch 8 | Train loss: 0.020986 | Val loss 0.028729
Epoch 9 | Train loss: 0.017214 | Val loss 0.024835
Epoch 10 | Train loss: 0.018561 | Val loss 0.038733
Epoch 11 | Train loss: 0.017493 | Val loss 0.020770
Epoch 12 | Train loss: 0.008237 | Val loss 0.017349
Epoch 13 | Train loss: 0.005149 | Val loss 0.013267
Epoch 14 | Train loss: 0.006774 | Val loss 0.019120
Epoch 15 | Train loss: 0.007481 | Val loss 0.014373
Epoch 16 | Train loss: 0.003869 | Val loss 0.015374
Epoch 17 | Train loss: 0.003134 | Val loss 0.026414
Epoch 18 | Train loss: 0.004332 | Val loss 0.015742
Epoch 19 | Train loss: 0.004408 | Val loss 0.016294
Epoch 20 | Train loss: 0.002298 | Val loss 0.011664
Epoch 21 | Train loss: 0.002010 | Val loss 0.011788
Epoch 22 | Train loss: 0.001391 | Val loss 0.010979
Epoch 23 | Train loss: 0.001266 | Val loss 0.010547
Epoch 24 | Train loss: 0.000736 | Val loss 0.010512
Epoch 25 | Train loss: 0.000582 | Val loss 0.010569

Training model: hidden_size=32, num_heads=2, num_layers=4
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid32_layers4_h
eads2.pt onto mps device.
Epoch 1 | Train loss: 0.626203 | Val loss 0.226660
```

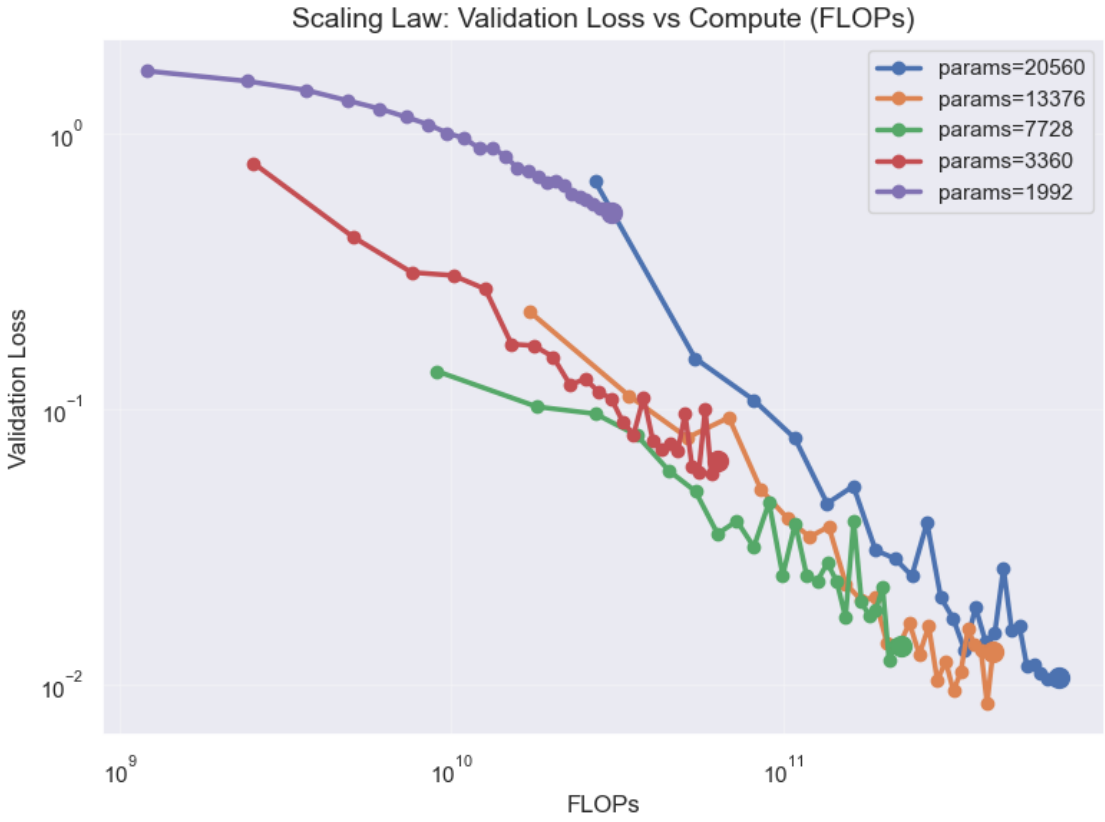
```
Epoch 2 | Train loss: 0.158771 | Val loss 0.111884
Epoch 3 | Train loss: 0.091994 | Val loss 0.078803
Epoch 4 | Train loss: 0.067058 | Val loss 0.093248
Epoch 5 | Train loss: 0.042171 | Val loss 0.050964
Epoch 6 | Train loss: 0.034123 | Val loss 0.040262
Epoch 7 | Train loss: 0.026373 | Val loss 0.034361
Epoch 8 | Train loss: 0.022235 | Val loss 0.037488
Epoch 9 | Train loss: 0.020471 | Val loss 0.023280
Epoch 10 | Train loss: 0.017970 | Val loss 0.020324
Epoch 11 | Train loss: 0.015300 | Val loss 0.020693
Epoch 12 | Train loss: 0.010333 | Val loss 0.014092
Epoch 13 | Train loss: 0.006809 | Val loss 0.013733
Epoch 14 | Train loss: 0.009267 | Val loss 0.016764
Epoch 15 | Train loss: 0.008585 | Val loss 0.012869
Epoch 16 | Train loss: 0.007558 | Val loss 0.016300
Epoch 17 | Train loss: 0.004313 | Val loss 0.010345
Epoch 18 | Train loss: 0.004013 | Val loss 0.012181
Epoch 19 | Train loss: 0.004019 | Val loss 0.009493
Epoch 20 | Train loss: 0.003804 | Val loss 0.011114
Epoch 21 | Train loss: 0.003044 | Val loss 0.015951
Epoch 22 | Train loss: 0.004143 | Val loss 0.013995
Epoch 23 | Train loss: 0.004601 | Val loss 0.013356
Epoch 24 | Train loss: 0.002478 | Val loss 0.008538
Epoch 25 | Train loss: 0.003044 | Val loss 0.013154

Training model: hidden_size=24, num_heads=2, num_layers=4
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid24_layers4_h
eads2.pt onto mps device.
Epoch 1 | Train loss: 0.332824 | Val loss 0.137187
Epoch 2 | Train loss: 0.136318 | Val loss 0.102007
Epoch 3 | Train loss: 0.098672 | Val loss 0.096196
Epoch 4 | Train loss: 0.077745 | Val loss 0.080298
Epoch 5 | Train loss: 0.061763 | Val loss 0.059715
Epoch 6 | Train loss: 0.048399 | Val loss 0.050352
Epoch 7 | Train loss: 0.044509 | Val loss 0.035286
Epoch 8 | Train loss: 0.037407 | Val loss 0.039287
Epoch 9 | Train loss: 0.032497 | Val loss 0.031630
Epoch 10 | Train loss: 0.027827 | Val loss 0.045814
Epoch 11 | Train loss: 0.026918 | Val loss 0.024948
Epoch 12 | Train loss: 0.022797 | Val loss 0.038503
Epoch 13 | Train loss: 0.021761 | Val loss 0.024778
Epoch 14 | Train loss: 0.018936 | Val loss 0.023858
Epoch 15 | Train loss: 0.017815 | Val loss 0.027666
Epoch 16 | Train loss: 0.017429 | Val loss 0.023673
Epoch 17 | Train loss: 0.015927 | Val loss 0.017590
Epoch 18 | Train loss: 0.013313 | Val loss 0.039375
Epoch 19 | Train loss: 0.014372 | Val loss 0.019983
```

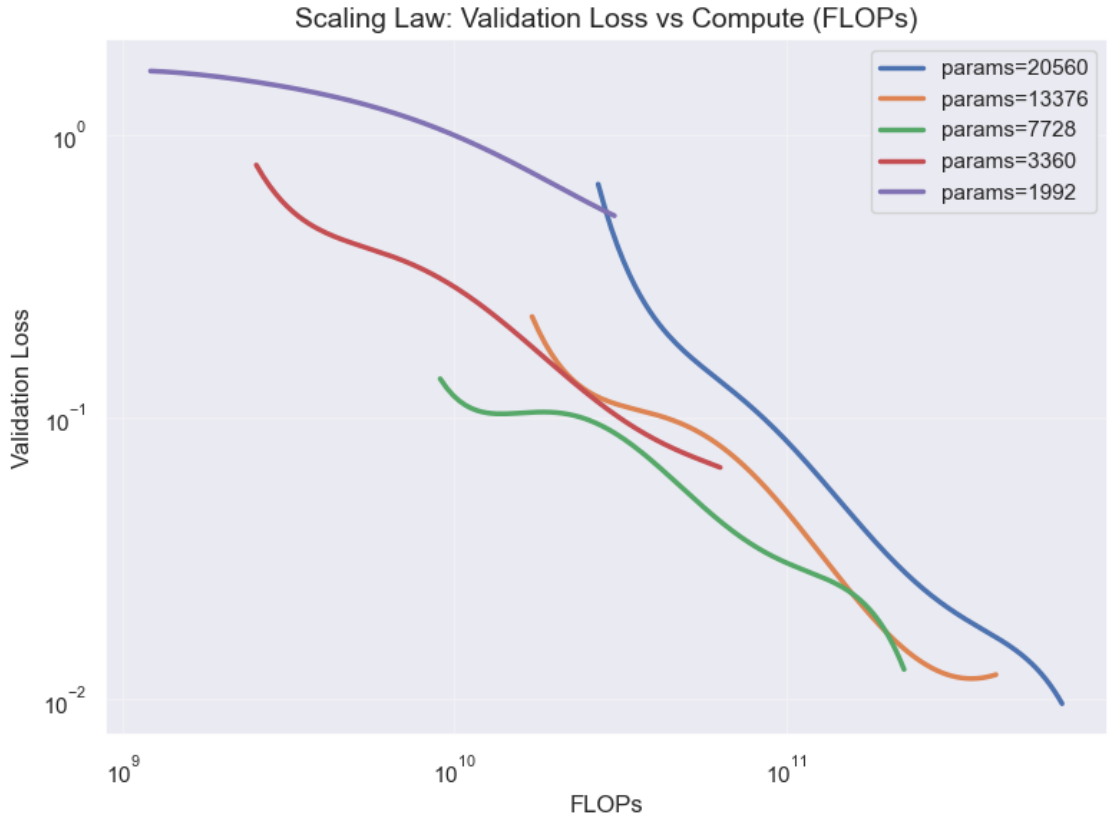
```
Epoch 20 | Train loss: 0.012826 | Val loss 0.017868
Epoch 21 | Train loss: 0.011387 | Val loss 0.018590
Epoch 22 | Train loss: 0.010276 | Val loss 0.022637
Epoch 23 | Train loss: 0.009457 | Val loss 0.012275
Epoch 24 | Train loss: 0.008262 | Val loss 0.013928
Epoch 25 | Train loss: 0.007551 | Val loss 0.013872

Training model: hidden_size=16, num_heads=1, num_layers=4
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid16_layers4_h
eads1.pt onto mps device.
Epoch 1 | Train loss: 0.298407 | Val loss 0.130236
Epoch 2 | Train loss: 0.084526 | Val loss 0.075383
Epoch 3 | Train loss: 0.073439 | Val loss 0.072686
Epoch 4 | Train loss: 0.074884 | Val loss 0.318166
Epoch 5 | Train loss: 0.065450 | Val loss 0.056413
Epoch 6 | Train loss: 0.062733 | Val loss 0.046585
Epoch 7 | Train loss: 0.057245 | Val loss 0.046918
Epoch 8 | Train loss: 0.051880 | Val loss 0.058239
Epoch 9 | Train loss: 0.049230 | Val loss 0.066891
Epoch 10 | Train loss: 0.043765 | Val loss 0.094727
Epoch 11 | Train loss: 0.042195 | Val loss 0.042310
Epoch 12 | Train loss: 0.036860 | Val loss 0.048052
Epoch 13 | Train loss: 0.034815 | Val loss 0.041075
Epoch 14 | Train loss: 0.033059 | Val loss 0.057601
Epoch 15 | Train loss: 0.031777 | Val loss 0.039322
Epoch 16 | Train loss: 0.031798 | Val loss 0.036972
Epoch 17 | Train loss: 0.029104 | Val loss 0.043511
Epoch 18 | Train loss: 0.028445 | Val loss 0.033316
Epoch 19 | Train loss: 0.025779 | Val loss 0.032366
Epoch 20 | Train loss: 0.023755 | Val loss 0.027126
Epoch 21 | Train loss: 0.021677 | Val loss 0.033200
Epoch 22 | Train loss: 0.021191 | Val loss 0.028775
Epoch 23 | Train loss: 0.020917 | Val loss 0.028452
Epoch 24 | Train loss: 0.020547 | Val loss 0.027248
Epoch 25 | Train loss: 0.018630 | Val loss 0.022767

Training model: hidden_size=16, num_heads=1, num_layers=3
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid16_layers3_h
eads1.pt onto mps device.
Epoch 1 | Train loss: 1.068045 | Val loss 0.781454
Epoch 2 | Train loss: 0.550112 | Val loss 0.421194
Epoch 3 | Train loss: 0.374141 | Val loss 0.313414
Epoch 4 | Train loss: 0.287023 | Val loss 0.305681
Epoch 5 | Train loss: 0.236598 | Val loss 0.272777
Epoch 6 | Train loss: 0.198353 | Val loss 0.172080
Epoch 7 | Train loss: 0.172094 | Val loss 0.170062
```

```
Epoch 8 | Train loss: 0.156113 | Val loss 0.154789
Epoch 9 | Train loss: 0.137031 | Val loss 0.122783
Epoch 10 | Train loss: 0.124263 | Val loss 0.128503
Epoch 11 | Train loss: 0.110700 | Val loss 0.115411
Epoch 12 | Train loss: 0.102132 | Val loss 0.108959
Epoch 13 | Train loss: 0.091151 | Val loss 0.089639
Epoch 14 | Train loss: 0.086317 | Val loss 0.080951
Epoch 15 | Train loss: 0.080013 | Val loss 0.110410
Epoch 16 | Train loss: 0.076733 | Val loss 0.076909
Epoch 17 | Train loss: 0.070704 | Val loss 0.071879
Epoch 18 | Train loss: 0.063811 | Val loss 0.075305
Epoch 19 | Train loss: 0.061596 | Val loss 0.070365
Epoch 20 | Train loss: 0.058951 | Val loss 0.096296
Epoch 21 | Train loss: 0.054408 | Val loss 0.061742
Epoch 22 | Train loss: 0.050902 | Val loss 0.059201
Epoch 23 | Train loss: 0.048161 | Val loss 0.099926
Epoch 24 | Train loss: 0.046163 | Val loss 0.058247
Epoch 25 | Train loss: 0.045451 | Val loss 0.064608

Training model: hidden_size=12, num_heads=1, num_layers=3
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid12_layers3_h
eads1.pt onto mps device.
Epoch 1 | Train loss: 1.606250 | Val loss 1.486954
Epoch 2 | Train loss: 1.340124 | Val loss 1.207574
Epoch 3 | Train loss: 1.038600 | Val loss 0.861892
Epoch 4 | Train loss: 0.782728 | Val loss 0.707117
Epoch 5 | Train loss: 0.634717 | Val loss 0.558059
Epoch 6 | Train loss: 0.537351 | Val loss 0.553718
Epoch 7 | Train loss: 0.479854 | Val loss 0.460927
Epoch 8 | Train loss: 0.434182 | Val loss 0.491324
Epoch 9 | Train loss: 0.417915 | Val loss 0.442564
Epoch 10 | Train loss: 0.376191 | Val loss 0.378826
Epoch 11 | Train loss: 0.341910 | Val loss 0.331617
Epoch 12 | Train loss: 0.324967 | Val loss 0.320721
Epoch 13 | Train loss: 0.302648 | Val loss 0.366901
Epoch 14 | Train loss: 0.288819 | Val loss 0.289654
Epoch 15 | Train loss: 0.272418 | Val loss 0.285745
Epoch 16 | Train loss: 0.259177 | Val loss 0.251023
Epoch 17 | Train loss: 0.241572 | Val loss 0.229630
Epoch 18 | Train loss: 0.225692 | Val loss 0.279837
Epoch 19 | Train loss: 0.215638 | Val loss 0.230677
Epoch 20 | Train loss: 0.205868 | Val loss 0.199519
Epoch 21 | Train loss: 0.201496 | Val loss 0.212249
Epoch 22 | Train loss: 0.183242 | Val loss 0.180133
Epoch 23 | Train loss: 0.175283 | Val loss 0.235717
Epoch 24 | Train loss: 0.170182 | Val loss 0.174715
Epoch 25 | Train loss: 0.164392 | Val loss 0.159902
```

```
Training model: hidden_size=12, num_heads=1, num_layers=2
[INFO] Loading model weights from /Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/MultiheadDecoderTransformer/Model_vocab30_hid12_layers2_h
eads1.pt onto mps device.
Epoch 1 | Train loss: 1.785961 | Val loss 1.685956
Epoch 2 | Train loss: 1.617972 | Val loss 1.548851
Epoch 3 | Train loss: 1.492224 | Val loss 1.437632
Epoch 4 | Train loss: 1.381100 | Val loss 1.317770
Epoch 5 | Train loss: 1.287916 | Val loss 1.231228
Epoch 6 | Train loss: 1.197418 | Val loss 1.149385
Epoch 7 | Train loss: 1.117825 | Val loss 1.080033
Epoch 8 | Train loss: 1.048070 | Val loss 1.004309
Epoch 9 | Train loss: 0.981025 | Val loss 0.961329
Epoch 10 | Train loss: 0.922533 | Val loss 0.883386
Epoch 11 | Train loss: 0.867679 | Val loss 0.886065
Epoch 12 | Train loss: 0.821292 | Val loss 0.824073
Epoch 13 | Train loss: 0.779774 | Val loss 0.752169
Epoch 14 | Train loss: 0.742627 | Val loss 0.732572
Epoch 15 | Train loss: 0.712286 | Val loss 0.699841
Epoch 16 | Train loss: 0.682646 | Val loss 0.668081
Epoch 17 | Train loss: 0.657836 | Val loss 0.670059
Epoch 18 | Train loss: 0.635605 | Val loss 0.646111
Epoch 19 | Train loss: 0.615669 | Val loss 0.601949
Epoch 20 | Train loss: 0.592914 | Val loss 0.589267
Epoch 21 | Train loss: 0.578662 | Val loss 0.573499
Epoch 22 | Train loss: 0.561698 | Val loss 0.557479
Epoch 23 | Train loss: 0.547712 | Val loss 0.537994
Epoch 24 | Train loss: 0.532104 | Val loss 0.532714
Epoch 25 | Train loss: 0.515401 | Val loss 0.517307
```

[39]:
```
plot_scaling_law(all_results)

plot_scaling_law_poly(all_results)
```


Scaling Law: Validation Loss vs Compute (FLOPs)
params=20560
params=13376
params=7728
params=3360
params=1992
Validation Loss
10^{0}
10^{-1}
10^{-2}
10^{9}
10^{10}
10^{11}
FLOPs



3.2 Step 2: Optimal Number of Parameters

Using the previously created training runs, we'll fit quadratic functions to the (parameter count, validation loss) pairs to estimate the optimal number of parameters for a given compute budget. We'll then fit a line to this set of optimal parameters to estimate the optimal number of parameters for a compute budget of 1e15.

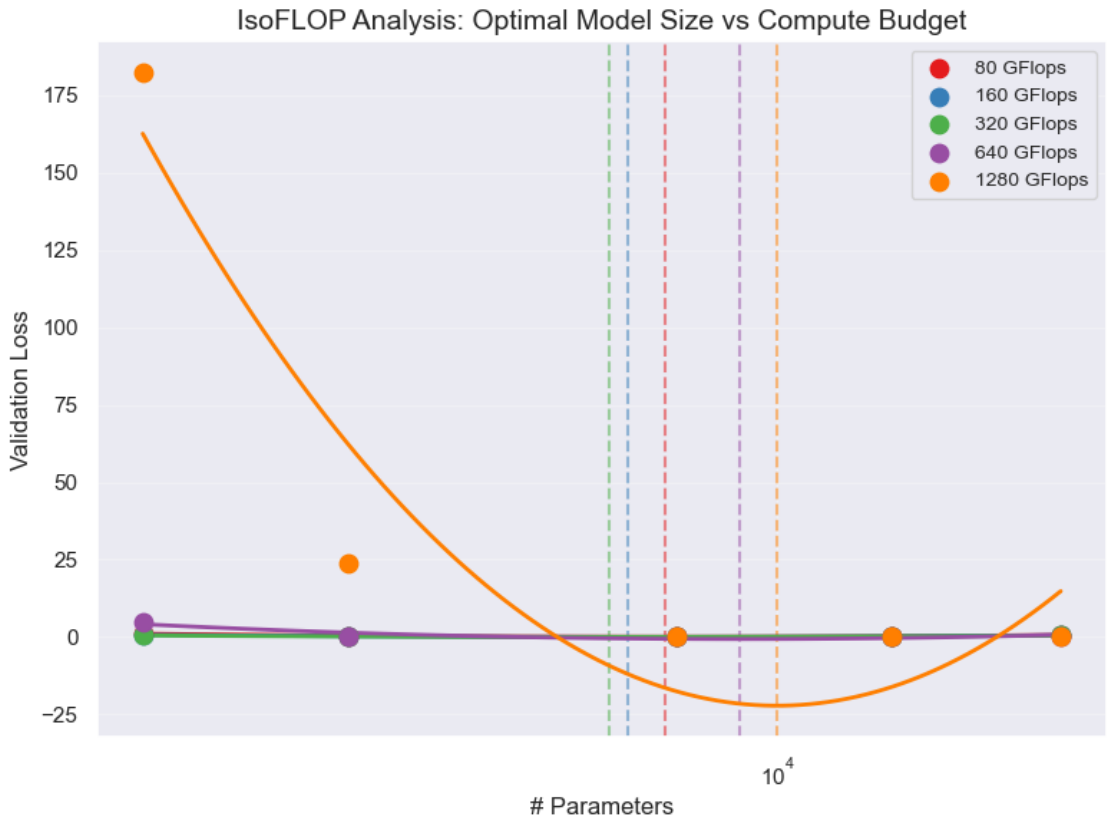
[40]:
```python
def find_optim_params(params: torch.Tensor, loss: torch.Tensor):
    x = np.array(params.cpu())
    y = np.array(loss.cpu())

    p = np.polyfit(np.log10(x), y, 2)
    optimal_log_params = -(p[1]) / (2 * p[0])
    optimal_params = 10 ** (optimal_log_params)

    return optimal_params


target_flops = [8e9, 16e9, 32e9, 64e9, 128e9]
colors = ["#E41A1C", "#377EB8", "#4DAF4A", "#984EA3", "#FF7F00"]
params_vs_loss = []
```

```
optimal_params = []

for target in target_flops:
    parameters = []
    val_loss = []

    for params, results in all_results.items():
        loss = interpolate(results["val_loss"], results["flops"], target, deg=4)

        # interpolate returns None if target is out of range
        if loss != None:
            parameters.append(params)
            val_loss.append(loss)

    sorted_indices = torch.argsort(torch.tensor(parameters)).to(device)
    parameters = torch.tensor(parameters).to(device)[sorted_indices]
    val_loss = torch.tensor(val_loss).to(device)[sorted_indices]
    try:
        optimal_params.append(find_optim_params(parameters, val_loss))
        params_vs_loss.append((parameters.tolist(), val_loss.tolist()))
    except Exception as e:
        print(
            ␣
 ↪f"{optimal_params=}\n\n{params_vs_loss=}\n\n{parameters=}\n\n{val_loss=}\n\n{target=}"
        )
```

```
/var/folders/dj/k5kk5wk90vvcnb_wphpw490r0000gn/T/ipykernel_92315/2694499551.py:2
: DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so
passing copy=False failed. __array__ must implement 'dtype' and 'copy' keyword
arguments. To learn more, see the migration guide
https://numpy.org/devdocs/numpy_2_0_migration_guide.html#adapting-to-changes-in-
the-copy-keyword
  x = np.array(params.cpu())
/var/folders/dj/k5kk5wk90vvcnb_wphpw490r0000gn/T/ipykernel_92315/2694499551.py:3
: DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so
passing copy=False failed. __array__ must implement 'dtype' and 'copy' keyword
arguments. To learn more, see the migration guide
https://numpy.org/devdocs/numpy_2_0_migration_guide.html#adapting-to-changes-in-
the-copy-keyword
  y = np.array(loss.cpu())
```

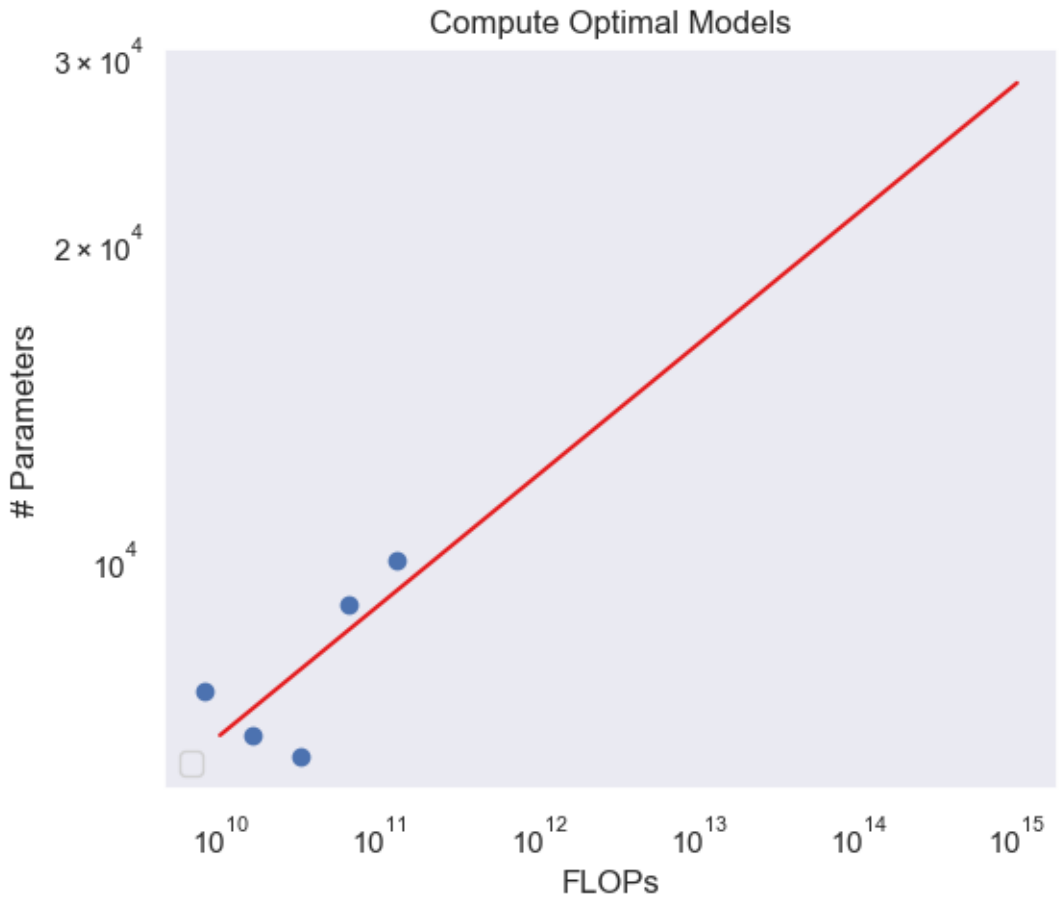
[41]:
```
plot_isoflop(target_flops, colors, params_vs_loss, optimal_params)
```

```
8.00 GFlops: Optimal params = 7,514
16.00 GFlops: Optimal params = 6,831
32.00 GFlops: Optimal params = 6,524
64.00 GFlops: Optimal params = 9,090
128.00 GFlops: Optimal params = 9,992
```



[42]:
```
plot_flops_params(target_flops, optimal_params)
```

```
y = 0.12346471733672994x + 2.59959596291159

/Users/Akseldkw/Desktop/Columbia/COMS4776/homework/HW2/utils.py:282:
UserWarning: No artists with labels found to put in legend.  Note that artists
whose label start with an underscore are ignored when legend() is called with no
argument.
  ax.legend(loc="lower left")
```



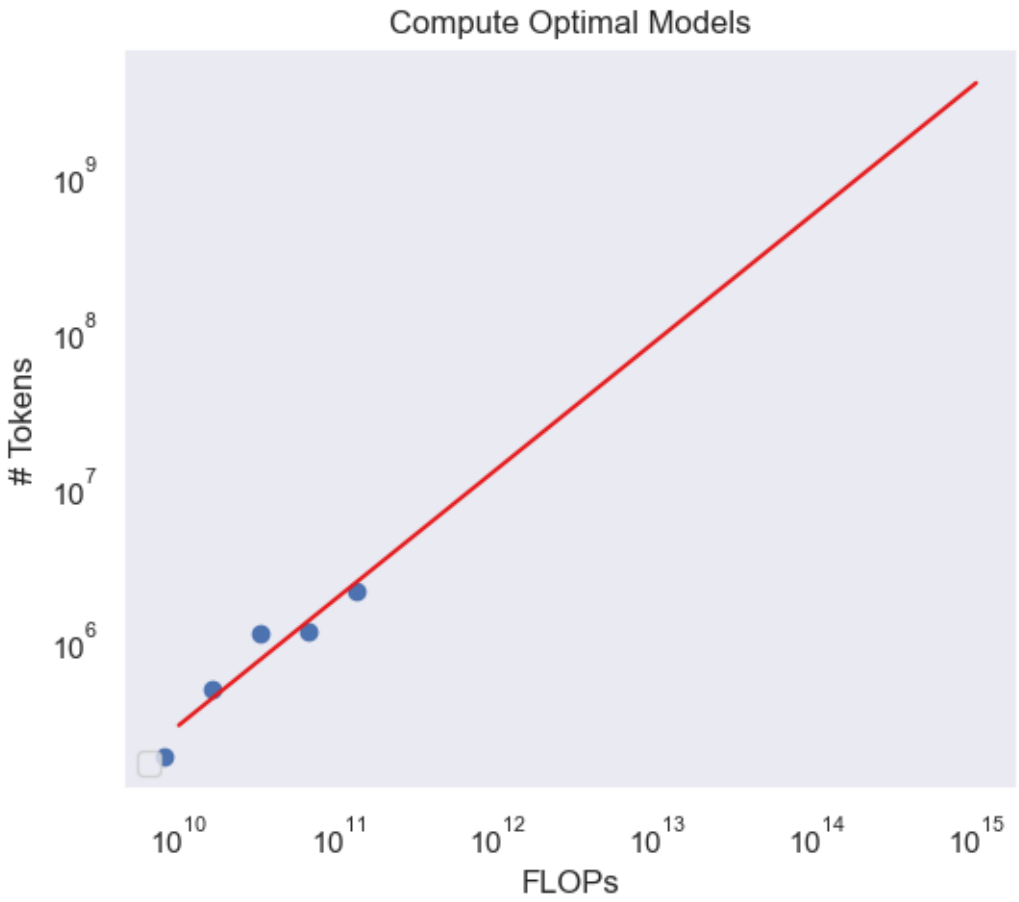
3.3 Step 3: Optimal Number of Tokens

Finally, we'll perform a similar analysis to the previous step, only this time determining the optimal number of tokens for a budget of 1e15 flops.

[43]:
```
plot_flops_tokens(target_flops, optimal_params, all_results)
```

```
y = 0.8263812499134265x + -2.804966406377365
```

```
/Users/Akseldkw/Desktop/Columbia/COMS4776/homework/HW2/utils.py:323:
UserWarning: No artists with labels found to put in legend.  Note that artists
whose label start with an underscore are ignored when legend() is called with no
argument.
  ax.legend(loc="lower left")
```


Compute Optimal Models
Tokens
10^9
10^8
10^7
10^6
10^{10}
10^{11}
10^{12}
10^{13}
10^{14}
10^{15}
FLOPs

[ ]: